PHASE 4 — Agentic Research Orchestration

Objective

Transform the basic RAG application into an agentic research assistant.

The agent must plan research tasks, dynamically select tools, execute them, and coordinate the research process.

Do not rewrite the existing ingestion, embedding, retrieval, or RAG components unnecessarily.

Step 1 — Define the agent architecture

Design a controlled agent execution flow.

The agent should contain clearly separated responsibilities:

- Planner
- Tool definitions
- Tool executor
- Agent execution loop
- Answer generation

Explain the data flow before implementation.

Step 2 — Implement planning

The planner receives a user question.

For complex questions, it should generate a concise, structured research plan.

Example:

Question:
"Compare warfarin and apixaban contraindications."

Possible tasks:

- Research warfarin contraindications.
- Research apixaban contraindications.
- Compare the retrieved evidence.
- Identify limitations.

The plan should be actionable and structured.

Do not expose private chain-of-thought.

Step 3 — Define local tools

Implement at least two local tools.

Tool A: search_corpus

Purpose:
Search the in-memory vector index for relevant passages.

Arguments should support:

- Search query.
- Top-K result limit, within validated bounds.

Return:
Relevant passages with source metadata and similarity information.

Tool B: read_source_document

Purpose:
Read a source document or relevant section for additional context.

Arguments should support:

- Source filename.
- Optional section or relevant location.

Security requirements:

- Restrict access to the configured corpus directory.
- Prevent path traversal.
- Validate filenames.
- Never accept arbitrary filesystem paths from the LLM.

Step 4 — Native tool calling

Use the LLM's native tool/function-calling capabilities.

The model should decide:

- Whether it needs a tool.
- Which tool to invoke.
- What arguments to supply.

The Python application must:

1. Receive the tool call.
2. Validate the tool name.
3. Validate arguments.
4. Execute the corresponding local function.
5. Return the tool result to the model.
6. Continue the execution loop when appropriate.

A Python dispatcher mapping tool names to functions is acceptable. It must not hardcode the research workflow or force every tool to run for every question.

Step 5 — Execution safety

Implement:

- Maximum tool-call/iteration limits.
- Argument validation.
- Unknown-tool handling.
- Tool execution error handling.
- Timeouts where appropriate.
- Clear execution state.
- Prevention of infinite loops.

Do not allow the LLM to execute arbitrary Python or shell commands.

Step 6 — Tests

Test:

- Simple factual questions.
- Multi-part research questions.
- Tool selection.
- Invalid tool arguments.
- Unknown tools.
- Tool failures.
- Iteration limits.
- Cases where the agent needs no additional tool.
- Cases where multiple tools are useful.

Mock LLM tool-call responses.

Constraints

- Do not add LangChain or LangGraph without a documented reason.
- Do not expose private chain-of-thought.
- Do not execute arbitrary code generated by the model.

- Do not rewrite the retrieval subsystem without necessity.
- Do not implement the critic in this phase.

Deliverables

Explain:

1. How planning works.
2. How the LLM chooses tools.
3. How the Python dispatcher works.
4. How tool results return to the model.
5. How execution limits protect the application.
6. Test results.